

Xây dựng, vận dụng và cải tiến đồ thị Trie

Wang Yun

2006

Nguồn gốc: Xây dựng, vận dụng và cải tiến đồ thị Trie

IOI China candidate team papers / 2006 / Wang Yun.doc

1 Từ cây Trie tới đồ thị Trie

Cây trie là một cách lưu từ điển. Mỗi từ tương ứng với một đường đi bắt đầu từ gốc; các ký tự trên cạnh của đường đi ghép lại thành từ đó. Cấu trúc này rất thuận tiện cho các thao tác có bản chất tiền tố, chẳng hạn tìm tiền tố chung dài nhất của hai từ thông qua tổ tiên chung gần nhất của hai nút.

Nếu bổ sung thêm các cạnh chuyển trạng thái, cây trie trở thành một **đồ thị trie**. Đồ thị này chính là mô hình quen thuộc phía sau thuật toán đa mẫu Aho–Corasick: khi đang đọc văn bản, mỗi ký tự đưa ta từ trạng thái hiện tại sang trạng thái kế tiếp, kể cả khi cây trie gốc không có cạnh trực tiếp mang ký tự ấy.

1.1 Ví dụ: bộ dò từ cấm

Cho một từ điển gồm các từ cấm, mỗi từ chỉ chứa chữ thường. Sau đó cho một đoạn văn bản, mỗi dòng cũng chỉ chứa chữ thường. Cần xác định liệu văn bản có chứa bất kỳ từ cấm nào hay không. Ví dụ, nếu rob là từ cấm thì problem chứa từ cấm ấy.

Với một dòng văn bản s dài L , cách nhìn trực tiếp là xét mọi tiền tố $s[1..i]$ và hỏi nó có kết thúc bằng một từ cấm hay không. Ta muốn kết quả ở bước i được suy ra nhanh từ trạng thái ở bước $i - 1$. Vì cạnh trên trie giống như biển chỉ đường theo ký tự, ta bắt đầu từ gốc và lần lượt đi theo $s_1, s_2, \dots, s_L$.

Vấn đề xuất hiện khi từ trạng thái hiện tại không có cạnh mang ký tự cần đọc. Ta không nên tạo một nút mới vô nghĩa; thay vào đó phải chuyển tới trạng thái đã tồn tại và tương đương về mặt hậu tố.

Với mỗi nút x , gọi chuỗi trên đường từ gốc tới x là **chuỗi đường đi** của x . Nút x là **nguy hiểm** nếu chuỗi đường đi của nó kết thúc bằng một từ cấm; ngược lại nó là an toàn. Với nút không phải gốc, x nguy hiểm khi chính chuỗi đường đi là một từ cấm, hoặc hậu tố bỏ ký tự đầu của chuỗi ấy tương ứng với một nút nguy hiểm.

Do đó ta cần **liên kết hậu tố** của mỗi nút: trạng thái ứng với hậu tố dài nhất đúng nghĩa của chuỗi đường đi hiện tại. Gốc trở về chính nó; các nút ở tầng một trở về gốc. Nếu nút v có cha p và cạnh từ p tới v mang ký tự c , thì liên kết hậu tố của v là trạng thái thu được khi từ liên kết hậu tố của p đi theo ký tự c .

Nếu trạng thái ấy chưa có cạnh c , ta tiếp tục theo liên kết hậu tố. Khi về tới gốc mà vẫn không có cạnh c , chuyển trạng thái về gốc. Như vậy các cạnh mới không tạo thêm chuỗi mới; chúng chỉ mô phỏng việc bỏ bớt tiền tố không còn hữu ích.

Thực tế, ta xây đồ thị trie bằng duyệt BFS trên cây trie:

1. tính liên kết hậu tố của mỗi nút;
2. truyền trạng thái nguy hiểm qua liên kết hậu tố;
3. bổ sung hoặc mô phỏng các cạnh còn thiếu theo quy tắc hậu tố.

Khi đọc văn bản, ta đi từng ký tự trong đồ thị. Nếu đi tới nút nguy hiểm, văn bản chứa từ cấm; nếu đọc hết mà không gặp nút nguy hiểm, văn bản an toàn.

Nếu tổng độ dài từ cấm là L_1 , tổng độ dài văn bản là L_2 , và kích thước bảng chữ là a , cách lưu đầy đủ mọi cạnh có độ phức tạp

$$\mathcal{O}(L_1 a + L_2)$$

về thời gian và $\mathcal{O}(L_1 a)$ về bộ nhớ.

2 Vận dụng thông tin trên đồ thị Trie

Đồ thị trie không chỉ lưu trạng thái nguy hiểm. Tùy bài toán, ta có thể lưu nguồn gây nguy hiểm, số lần trạng thái được ghé thăm, hoặc dùng thứ tự BFS để đẩy thông tin ngược qua liên kết hậu tố.

2.1 Ví dụ: ô chữ

Bài *WPUZZLES* cho một ma trận $L \times C$ chữ hoa và W từ. Mỗi từ xuất hiện đúng một lần trong ma trận, theo một trong tám hướng. Cần tìm vị trí chữ cái đầu và hướng của từng từ.

Mô hình đa mẫu rất rõ ràng. Vì đề hỏi vị trí chữ cái đầu, ta đưa các từ vào trie theo thứ tự đảo ngược. Chẳng hạn, *MAIGO* được đưa vào dưới dạng *OGIAM*. Sau đó, với mỗi hướng và mỗi đường thẳng trong ma trận, ta chạy khớp đa mẫu một lần.

Trong bài này, chỉ biết một nút có nguy hiểm hay không là chưa đủ. Ta cần biết nút nguy hiểm do từ nào gây ra. Nếu chuỗi đường đi của nút x chính là một từ, nguồn nguy hiểm của x là từ ấy; nếu không, nguồn nguy hiểm được kế thừa từ liên kết hậu tố của x .

Khi gặp một nút nguy hiểm, không thể chỉ ghi nhận một từ rồi dừng. Một trạng thái có thể đồng thời kết thúc nhiều mẫu, ví dụ một từ là hậu tố của từ khác. Vì vậy sau khi ghi nhận nguồn nguy hiểm hiện tại, ta phải tiếp tục lần theo liên kết hậu tố tới khi gặp nút an toàn, ghi nhận tất cả các từ kết thúc tại đó.

2.2 Đếm số lần xuất hiện

Trong một số bài, ta không chỉ muốn biết từ nào xuất hiện mà còn cần đếm số lần xuất hiện của từng mẫu. Khi quét văn bản, ta tăng bộ đếm của trạng thái đang đứng. Nhưng bộ đếm tại nút ứng với một từ chưa chắc bằng số lần xuất hiện của từ ấy, vì con trỏ có thể đang ở trạng thái dài hơn mà từ cần đếm là hậu tố.

Cách xử lý là dùng thứ tự BFS. Sau khi quét xong, duyệt các nút theo thứ tự độ sâu giảm dần và cộng số lần ghé thăm của mỗi nút vào liên kết hậu tố của nó. Sau bước này, bộ đếm tại nút ứng với một từ chính là số lần từ ấy xuất hiện.

3 Đồ thị an toàn

Trong nhiều bài, các nút nguy hiểm đóng vai trò chướng ngại. Những nút thật sự hữu ích là các nút an toàn mà đường từ gốc tới chúng chưa từng đi qua nút nguy hiểm. Bản gốc gọi tập nút và cạnh giữa chúng là **đồ thị an toàn**.

3.1 Ví dụ: mã an toàn vô hạn

Bài *Virus* của POI cho một tập các chuỗi nhị phân là mã đặc trưng của virus. Một chuỗi nhị phân không chứa mã virus nào là mã an toàn. Cần hỏi liệu có tồn tại một mã an toàn dài vô hạn hay không.

Từ gốc của đồ thị trie, việc đọc một mã an toàn tương ứng với việc đi trong đồ thị an toàn. Tồn tại mã an toàn dài vô hạn khi và chỉ khi trong đồ thị an toàn có chu trình đi được từ gốc. Vì vậy ta xây đồ thị trie, loại bỏ các nút nguy hiểm, rồi kiểm tra chu trình trong phần còn lại. Có thể dùng DFS đánh dấu hoặc thử topo; nếu topo thất bại thì có chu trình và đáp án là có.

3.2 Ví dụ: đếm chuỗi không chứa từ cấm

Bài *Censored!* hỏi số chuỗi độ dài m trên một bảng chữ kích thước n không chứa bất kỳ từ cấm nào. Với $m \leq 50$, số lượng có thể lớn nên cần số nguyên lớn.

Trên đồ thị an toàn, bài toán trở thành đếm số đường đi dài m bắt đầu từ gốc. Đặt

$$dp[t][x] = \text{số cách đi } t \text{ bước và kết thúc ở nút } x.$$

Khởi tạo $dp[0][\text{gốc}] = 1$. Với mỗi bước, duyệt mọi cạnh an toàn (u, v) và cộng

$$dp[t][v] += dp[t-1][u].$$

Tổng $\sum_x dp[m][x]$ trên các nút an toàn là đáp án. Đây là một ví dụ cho thấy đồ thị trie không chỉ dùng để khớp chuỗi, mà còn có thể làm không gian trạng thái cho quy hoạch động.

4 Cải tiến bộ nhớ

Nhược điểm của đồ thị trie lưu đầy đủ là bộ nhớ. Nếu bảng chữ lớn, mỗi nút phải lưu rất nhiều cạnh. Với 100000 nút và bảng chữ gần như toàn bộ ký tự ASCII, số cạnh đầy đủ đã vượt xa giới hạn bộ nhớ của nhiều bài.

4.1 Ví dụ: bộ lọc từ cấm

Bài *Ural 1269* cho một từ điển từ cấm có tổng kích thước khoảng 100 KB và văn bản khoảng 900 KB. Văn bản có thể chứa gần như mọi ký tự trừ ký tự kết thúc dòng. Cần tìm vị trí xuất hiện đầu tiên của một từ cấm, hoặc in *Passed*.

Nếu lưu đầy đủ cạnh từ mỗi nút cho mọi ký tự, bộ nhớ không chịu nổi. Điều quan trọng là các cạnh gốc của cây trie phải được lưu, nhưng các cạnh “bổ sung” có thể được tính khi cần.

Định nghĩa hàm $\text{child}(x, c)$:

- nếu x có con thật theo ký tự c , trả về con ấy;
- nếu không và x không phải gốc, thay x bằng liên kết hậu tố của nó rồi thử lại;
- nếu về tới gốc mà vẫn không có cạnh c , trả về gốc.

Hàm này thực hiện đúng quy tắc bổ sung cạnh của đồ thị trie, nhưng không cần lưu tất cả cạnh bổ sung.

Bộ nhớ vì vậy giảm từ $\mathcal{O}(L_1 a)$ xuống $\mathcal{O}(L_1)$. Thời gian vẫn là $\mathcal{O}(L_1 + L_2)$ theo phân tích khấu hao: khi xây liên kết hậu tố hoặc khi quét văn bản, độ sâu của con trỏ chỉ tăng từng bước theo cạnh thật, còn các lần nhảy hậu tố làm độ sâu giảm. Tổng số lần giảm không thể vượt quá tổng số lần tăng quá nhiều lần.

Cách cải tiến này về bản chất là cài đặt tiết kiệm bộ nhớ của thuật toán Aho–Corasick. Với bảng chữ nhỏ và cần duyệt mọi cạnh để topo hoặc quy hoạch động, lưu đầy đủ có thể nhanh hơn. Với bảng chữ lớn hoặc gần như không giới hạn, phiên bản tính cạnh khi cần là lựa chọn bắt buộc.

5 Kết luận

Đồ thị trie bắt đầu từ một ý tưởng rất đơn giản: biến các tiền tố trong từ điển thành trạng thái, rồi dùng liên kết hậu tố để xử lý khi khớp thất bại. Từ đó ta có một cấu trúc mạnh cho đa mẫu:

- khớp nhanh nhiều mẫu trong một văn bản;
- đếm số lần xuất hiện bằng cách đẩy bộ đếm qua liên kết hậu tố;
- xây đồ thị an toàn để kiểm tra chu trình hoặc làm quy hoạch động;
- đổi bộ nhớ lấy thời gian bằng cách tính cạnh bổ sung khi cần.

Phụ lục mã nguồn và các đường dẫn chương trình trong bản gốc không được chép lại, vì đây là các cài đặt mẫu dài; phân tích tập trung vào cấu trúc dữ liệu, cách xây dựng và các ứng dụng thuật toán.